

LECTURE - 1:

Tuesday.

20/8/24

- When high level lang. file like "test.cpp" is saved; it is saved in computer in form of "exe file" (executable file). This file will have binary values only. (machine code)

Games → .exe file is available for users to play & download
 Source code (C++ code) is confidential.

- Compiler first finds syntax errors in code. Then translates it in binary (low level lang). to run exe file.

LECTURE - 2

Thursday

22/8/24.

- Libraries have built in codes e.g.: iostream library.
 #include is reserved word for C++ to import library

Code $\xrightarrow{\text{sent to}}$ Preprocessor Directives $\xrightarrow{\text{sent to}}$ Compiler.

- This will filter the lines which have '#' in beginning since it indicates that libraries has to be imported.

Now a software called "Linker" will link code with these libraries as Preprocessor directives will send these lines to Linker.
 → this will be done after code has been gone through compiler.

- Standard Name space (std) ^{Stores the name labels.}

cout
cin
SQRT
Sum

std:: cout << "Hello world"
std:: cout << "Fast - No" X

In beginning of code ← using namespace std. ✓

- "<<" will be defined in <iostream> library with name "cout".
But the name label "cout" will be coming from namespace "std".

- Operating system executes the task.

- "Return 0" is for telling the OS that code is successfully runned. That's why it is written at end of code.

- System ("pause"); will run program if we'll click 'exe' file in 'MyPc'.

→ return type: the datatype which this function will return. Here

- int main(): it is 'int' because at last we use 'return 0'.
we've to tell OS since it is expecting us to return smth since OS called the function "main".

↑ void main(): doesn't return anything

LECTURE-3

Tuesday
27/8/24.

- Source code → Preprocessor directives → Object code
→ Compiler → Linker → Full Binary code (executable file).

- Object code is partially binary code which have the functions of code encoded in binary form.
Except the fact, things imported from libraries e.g: "cout/cin" won't be translated in binary.

- Then the Linker will import the libraries & convert the words like "cout" into binary.
- Then finally we'll get an executable file.
- Linker will first substitute the function we used like "cout" with its original code (of how's it work) from backed. Then, that substituted code will be converted into machine lang.

Code:

```
#include <iostream>
```

```
using namespace std;
```

```
namespace Fast {  
    string library = "Justice M Cul"  
}
```

```
namespace LUMS {  
    string library = "Syed Babar Ali"
```

```
int main() {  
    cout << Fast::library << endl;  
    cout << LUMS::library << endl;
```

Output Screen: Justice M Cul
Syed Babar Ali

necessary since we've two function named as "library".

Creating Custom Namespaces named as "Fast" and "LUMS".

- Datatype is necessary in order to tell RAM how much memory to occupy bcs datatype like 'int' allows us 32 bits to be stored.
- It also tells which operations we can perform on the data.
- Whenever variable is declared: int length, a box named 'length' of 4 bytes has been created in RAM and it will initially have some "garbage value" which will be replaced when we assign value to 'length'.

LECTURE-4: Thursday 29/8/24.

- Short int: 2 bytes
- int: 4 bytes
- long long int: 8 bytes
- float: 4 bytes
- double: 8 bytes

⇒ Code to add "9 and 27" from 9.27.

```
int main() {
    float v1 = 9.27
    int integerPart = v1
    float decimalPart = (v1 - integerPart) * 100
    cout << integerPart + decimalPart << endl
}
```

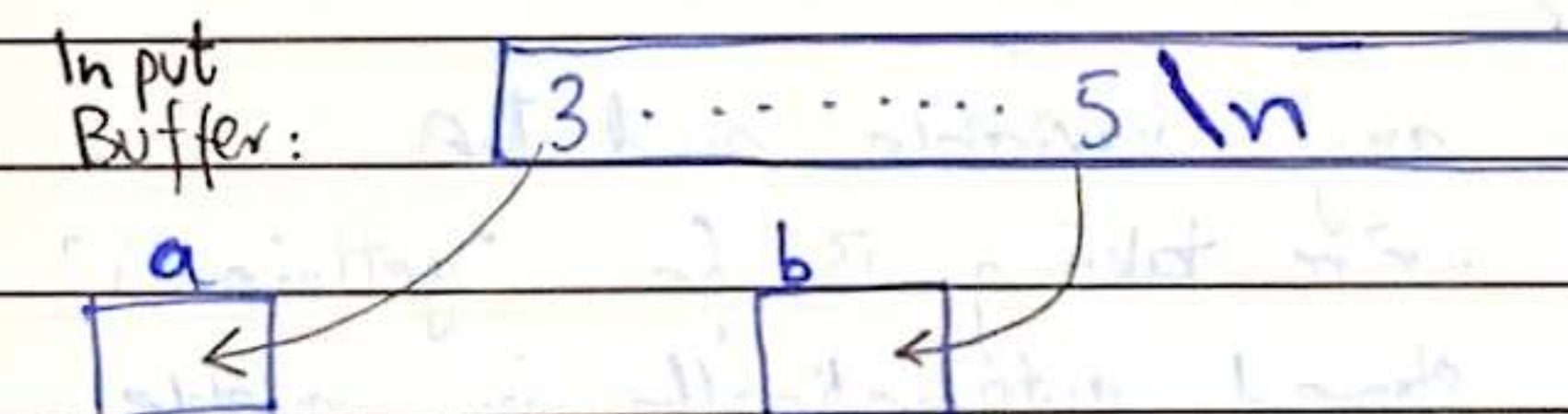
- "4/8" is integer division and gives answer "0"
- "4.0/8" is float division & gives answer "0.5"
- ↳ at least one operand should be float like '4.0' for float division

- "%" is for mod (modulus) → Returns remainder of 2 digits addition
- Modulus is used for calculations like "2+8" from 28.
- By ~~dividing~~ anything by 10, the quotient (integer part) will be "tenth place" and the remainder (after modulus) will be "units place".
- All variables in modulus should be "int". Ek bhi float hoga to answer wld be in decimals.

LECTURE-5: Tuesday 3/9/24.

- Max value that can be stored in any specified bits = $(2)^{N-1} - 1$ (where 'N' is no. of bits allocated to that datatype)

- When we input any data, it goes to "input buffer".
- ^{"Cin"} ignores "ln" and spaces before/after any data.



- If we have data in input buffer, the program won't ask for the next input since it can fetch data from buffer that time.

★ Same problem of not taking values after space is with 'int' and 'float' since we use them with "cin>>". But numbers can't be handled with "getline()".

Day: ☆☆☆☆☆☆☆

Date: □ □ □

Day: M T W T F S S
☆☆☆☆☆☆

Date: □ □ □

~~String name;~~
~~cout << "Enter name:";~~
~~cin >> name;~~
~~cout << "Your name is:" << name;~~

Output:

Enter name: Alyan Asif
Your name is Alyan

String names;

cout << "Enter name:";

cin >> name;

cout << "Your name is:" << name;

★ "cin>>" will stop taking data once we've space in between data.

→ To take input of data with spaces:

★ #include <string>

string name;

only for string inputs

★ getline(cin, name);

cout << "Your name is:" << name;

Output

Enter name: Alyan Asif
Your name is: Alyan Asif

• "cin>>" will stop getting data to store in variable as soon as it will find "spaces" or "new line character (\n)" (when we press Enter).

• getline(____) will empty the entire input buffer along with "spaces" and "\n".

~~therefore we should either~~

• If we do "cin>>" in any variable and then press enter. If the next input we're taking is from "getline()" function; then "\n" will be stored automatically in variable inside getline function and the program won't even ask for input of that variable.

• So in order to not store "\n" in variable inside variable of getline(); we can do 3 things:

1) Write all "getline()" functions first and then write all "cin>>" statements.

2) -Using "cin.ignore()" after "cin>>" statement will basically eat up/erase one character which could be either next line character (\n) after pressing enter or could be "space".
- Need to wait it again and again for more null characters removal

3) After "cin>>" if we've multiple "waste values" like: multiply spaces and enter presses (\n); then we can use "getline(cin, temp)" once after 'cin' in order to clear input buffer completely and then write actual getline() function after it.

Thurs day

5/9/24.

LECTURE 6:

⇒ Unary Operators:

• Addition, multiplication, ... : Binary operators (2)

• "++" : unary operator (to increment a variable's value by 1)

pre-increment ++x; OR x++; post-increment

"--" : For decrement of 1 from a variable

pre-decrement --x; OR x--; post-decrement

• In "pre-increment" operator, value of x is incremented first; then any BODMAS rule will be applied. ++ > *, /, +, -

• In "post-increment" operator, BODMAS rule will be applied first, then that will be final answer but it will also update the variable before "++" and increment by 1 later.

★ same for pre-decrement and post decrement.

All precedences and associativity in "files" of phone.

MTWTFSS
Day: ☆☆☆☆☆☆☆

When operators of same precedences are in one expression.
↑ Associativity. Date

Precedence/Importance

Working

Precedence

1 st	++x or --x	Right to left	If more than 1 operator of same importance.
2 nd	* / %	left to right	
3 rd	+ -	left to right	
4 th	x++ or x--	left to right	
5 th	==	right to left	
least	=	right to left	

* bool uses 1 bit (0 or 1) to check true/false.

* For assignment "=", for comparison "=="

* If $10 * 2 + 10 / 5$; then $(20) + (2) = \boxed{22}$

int score;

cin >> score;

if (score >= 50) {
cout << "Congrats";
}

else {
cout << "Better luck next time";
}

if (0) → will treat as false

if (1) → will treat as true

any integer (+, -)
other than 0

→ "else if" statement same as "else".

* if (score < 59 ~~AND~~ score > 50) {
}
if score < 59 ~~OR~~ score > 50 {
}

* If there isn't {} after 'if' statement then only the first statement in 'if' block will be executed.

* Koi bhi function horha ho (like: x=y, ++c/2 etc) inside if/else statement; wo phir bhi puray program pr apply hogi.

* int max = (a > b) ? a : b

Qls mark say phle "condition", qls mark kay baad (by colon) is the thing to select if true, colon kay baad the thing to select if false.

MTWTFSS
Day: ☆☆☆☆☆☆☆

Date

LECTURE - 7

Tuesday
10/9/24

⇒ Compound Operators.

jo $a += b$

equals to $a = a + b$

Kay LHS $a *= b$

pr variable $a /= b$

hoti; ans. ki value use $a ./= b$

main store hoti

$a = a + b$

$a = a - b$

$a = a * b$

$a = a / b$

$a = a ./ b$

$a * = b * = c \rightarrow ① b = b * c \quad ② a = a * b$

$a * = b + c \rightarrow a = a * (b + c)$

⇒ Precedence/Importance

+ , -
+= , -= , *= , /=

Compound operator lower than +, -

Associativity/Working.
left to right
right to left.

* $a = c * = b$ ✓

$a = (a - c) * = b$ X

When compound operators are used, you use wazif you can't do any other operation b/w the two "equals to" sign.

* The only thing that could be done is using any operator on the very right as associativity is right-to-left.

e.g: $a = c * = b - c$

⇒ "if/else" and "Nested if".

LECTURE - 8

Thursday
12/9/24

• "!=" → is not equal to

inverse/not ← 2nd

(after pre-increment)

→ logical Not operator: (!x) ↑ Highest Precedence ↑

int main() {

int x = 4;

int r = !x;

→ !x converts non zero value of 'x' to zero and zero value of 'x' to 1.

• But these change of values is just for variable 'r'. The value of 'x' inside memory will be unchanged (i.e. 4).

★ '!' kay baad koi bhi variable ya number likh saktay; it'll be inverted.

★ Ager if statement completely run hogai hai, to uskay and memory ander jo bhi operation hoga hoga; that will apply in whole program.

conditions

★ When there are more than one ~~statements~~ within "if" statement brackets (); then whenever ~~the~~ any condition becomes 'true'; uskay aagay ki conditions check nahi hon gayi neither will those conditions be applied to program or memory.

While () {

if () {

int a = 10;

}

• 'a' ki life sirf iss bracket kay ander hai

→ This variable 'a' could only be used in the sub-blocks of 'if' statement or the same block of 'if' statement.

• Cannot be changed or used outside the parent block.

• Even 'if' kay bahir ham 'a' ko print nahi kar sakte.

Thursday
19/9/24

LECTURE - 9

• While loops:

int main() {

int counter = 1;

while (counter <= 10) {

cout << "Cheezious";

counter++;

}

★ While (0) → Considered False

While (any numbers other than '0') → Considered True.

Q: Sum of Geometric Series: 1, 2, 4, 8, 16, 32, 64.

Ans: $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^5 + 2^6$

use while loop on power of 2 & increment it

int r;

cout << "Enter common ratio b/w terms of geometric series";

cin >> r;

int Sum = 0;

int Count = 0;

→ loop control variable

While (Count <= 6) {

Sum += pow(r, Count); → Sum = Sum + pow(r, Count)

Count++;

}

cout << Sum

LECTURE-10: Thursday 26/9/24

→ To reverse any number of n^{th} digit:
→ we don't know how many.

```
int num, rev=0;
cin >> num; // 328
while (num > 0) {
    rev = (rev * 10) + (num % 10); // 8 in 'rev' as (0*10)+(8)
    num = num / 10; // 328 → 32
}
cout << rev << endl;
```

⇒ FOR loop: loop control variable

★ we can declare/initialisation inside the parenthesis of FOR loop. A semi-colon is mandatory if you declare/initialize inside brackets.

```
for (;;)
{
    cout << "Fast PF" << " ";
}
```

logical error since no condition in For loop; makes it an 'infinite loop'.

→ this is optional (but could be used to initialise & declare loop control variable) and that LCV can't be used outside loop due to life of variable being just inside loop.

Syntax: `for (declare/initialise; condition; increment/decrement)`
→ this is compulsory

→ Print Geometric Series: 1, 2, 4, 8, 16

```
for (int count=0; count < 5; count++) {
    cout << pow(2, count);
}
```

Working of FOR loop:

- 1st: initialise
- 2nd: condition check
- 3rd: count increment
- 4th: condition check
- 5th: count increment

★ Initialisation / declaration will only occur once.

→ Print Arithmetic Series from '1' for next 10 terms with common difference: 3 (1, 4, 7, 10, ...).

```
int t = 1;
for (int i = 1; i <= 10; i++) {
    cout << t;
    t = t + 3;
}
```

the 'i' is loop control variable and will make sure the loop runs for 10 times.

• So here in brackets; we should just make sure that loop runs a certain times instead of incrementing 'i' by '3' & printing i.

OR

```
for (int t=1, i=1; i <= 10; i++, t+=3) {
    cout << t;
}
```

Here the key thing is that we can follow the basic syntax of "declare; condition; increment/decrement" and do anything with it BUT COMMA SEPERATED.
★ 'int' just have to be written once.

In for loop; if it is $i--$, then do $i \geq \text{value}$;
if it is $i++$, then do $i \leq \text{value}$;

M T W T F S S
Day: ☆☆☆☆☆☆☆

Date □ □ □

M T W T F S S
Day: ☆☆☆☆☆☆☆

or anywhere we're getting multiple inputs. Date □ □ □

LECTURE-11 Tuesday 1/10/24.

★ Nested FOR loop main phelay outer aik dafa chalay ga and then inner 7 times chalay ga (supposing inner ki iteration is till 7).

★ If outer loop is iterated for 3 times and inner is iterated for 5 times; then loops will work like this:

Outer Inner → Pencils say Chronological steps
likhain hain
① 1 5 6 7 8 9 10 11 12
② 2 5 6 7 8 9 10 11 12
③ 3 5 6 7 8 9 10 11 12
Total iterations = $3 \times 5 = 15$

★ Therefore whenever we need to print series like 1 2 3 or X X X X X
we need to link the "loop control variables" of both to outer and inner loops (i.e. 'i' and 'j' respectively).

★ WHILE & FOR loops are "pre test" loops.

⇒ DO-WHILE loops: Post test loop.

int counter = 5;

1 do {
2 cout << ---
3 }
4 while (counter < 10);

→ If condition true; the loop will repeatedly run upon lines: 1, 2, 3 and 4.
• Whenever the condition gets false; the loops fails and rest of the program gets executed normally.

★ If we do "input validation" (for +ve values or values in a specific range) using "WHILE" loop; we need to write "cin" statements two times.
Whereas; if we'll use "DO-WHILE", we can just use "cin" once.

for () {

for () {

for () {

Now in these 2 for loops; we can name the loop control variable similarly since the life of that same named variable in each for loop will remain alive just until that loop is scope is present.

STATIC

⇒ ARRAYS:

datatype

Name

size of array it will be like 0, 1, 2, 3, 4-
* n is size of array: n-1

- Creation: int Arr[5]
- Storing data: int Arr[5] = { 3, 6, -2, 10, 0 }
- Accessing Array data: Arr[2] → index position (cout << Arr[2])

int Numbers[5];

for (int i=0; i<5; i++) {
cin >> Numbers[i];
cout << Numbers[i];
}

int Arr[1]

for (int i=0; i<5; i++) {
cin >> Arr[i];
}

- Storing '0' in all indexes: here data will be over written on int Arr[5] = { 0 } array's index position "zero" again and again for 5 times since the size of array is "1 box".

If we don't know about length of any string or a specific type of string/sequence inside an Array, then we can use 2 variables → Max count
Current

M T W T F S S
Day: ☆☆☆☆☆☆

Date □ □ □

M T W T F S S
Day: ☆☆☆☆☆☆

Date □ □ □

★ Static array - key size main there must be a const value. It could be a numeric value; it could be a const variable; but it can't be a variable.

★ `int Arr[] = {2, 4, 100, 12}` → This is also static.
`for (int i = 0; i < sizeof(Arr) / sizeof(int); i++) {`
`cout << Arr[i];`

→ "sizeof(datatype)" returns the value of "bytes" allocated for any datatype.

LECTURE-13 Tuesday, 8/10/24.

`int smallest = INT_MIN;` This will assign the lowest value possible in integer datatype to variable "smallest".

• Best approach to find the : second, third, seventh or n^{th} largest value in an array is to sort it first →

Thursday 10/10/24.

LECTURE-14

→ Find the common elements in 2 arrays. (Intersection)

`int arr1[5] = {3, 2, 9, 7, 13}` `int arr2[6] = {3, 7, 0, 15, 11, 2}`

`for (int i = 0; i < 5; i++) {`

`for (int j = 0; j < 6; j++) {`

`if (arr1[i] == arr2[j]) {`

`cout << arr1[i] << " ";` check rest of array.

`break;` → needed since agar shuru

main he ek value common mil gayi; then don't

Imperial Notes

→ Code for Union of two arrays.

⇒ Sorting an Array: (Ascending)

`int arr[5] = {2, 9, 3, 1, -20}`

`for (int i = 0; i < 4; i++) {`

`for (int j = 0; j < 4; j++) {`

`if (arr[j] > arr[j+1]) {`

`temp = arr[j];`

`arr[j] = arr[j+1];`

`arr[j+1] = temp;`

The gist is key string ka har ek character will be stored at each index of char array. And last main humesha '\0' aayega.

★ In a character variable; we can only store one letter e.g "y" (for yes) and "n" (for no).

★ But in order to store entire string in character, we need:

→ Character Array: we can ask user for choice by giving them entire words as choice e.g "yes" or "no".

`char Array[] = {"Programming Fundamentals"};` can be done without {} and only " " could also be → `{ 'P', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g' }` comma separated

★ In character arrays, on the last cell of array; there is a "null terminator" (\0) stored therefore the memory of array should be 1 greater than the required memory.

`char Arr[3] = {"PF"};`

• P stored at 0th index

• F stored at 1st index

• \0 (null terminator) stored at 2nd index

Imperial Notes

* Strlen (variable name) is function to get length of string's ^{each} character.
 ↳ only for character array... it will count.
 Day: ☆☆☆☆☆☆ M T W T F S S • It won't count null terminator.

Date

* To print normal array; we need loop.
 But to print a character array, we can directly do it using "cout <<".
 cout << Name of Character Array << endl;

* "Character" datatype requires 1 byte
 "Integer" datatype requires 4 bytes.

* While assigning values in char array directly by:
 * Char Arr[] = "PF", the null terminator is automatically stored at the end (on 2nd index).
 * But if we'll use "cin >>" and while loop like this:

```
int i = 0;
char Arr[25];
while (i < 25) {
```

```
    cout << "Enter data : ";
    cin >> Arr[i];
    cout << Arr << endl;
```

and we'll only store two characters "PF"; then the rest of the space i.e. 22 (apart from 25th space which is for \0) will contain garbage value and output it. Matlab jesy he null terminator aaye ga, uska baad ki koi values output nahi hogi.

⇒ The right way to get "input" in character array:
 cin.getline (Arr, (Memory of Array))

Here we ~~use~~ ^{have to} use ^{exact} size of array but it will only take (size - 1) input characters as it also has to store '\0'.

* The values of array length, parameter in cin.getline() and in for loop for output will all be same.

Day: ☆☆☆☆☆☆ M T W T F S S

Date

⇒ ~~function~~ ^{code} to create "string's length" function:

```
int i = 0;
while (crr_Name[i] != '\0') {
    i++;
}
cout << "String length : " << i << endl;
```

* Declaration kay wagt ziada values than array length store karein so its ~~is~~ syntax error.
 * Inputting array length say ziada length ki value store krein tou run time error.

LECTURE-16 Thursday 17/10/24.

⇒ String Concatination:

```
int k = 0;
for (int i = 0; i < S1; i++) {
    c_Arr[k++] = c1[i];
}
c_Arr[k++] = " ";
for (int i = 0; i < S2; i++) {
    c_Arr[k++] = c2[i];
}
c_Arr[k] = '\0';
```

```
char c1[] = "Digital"
char c2[] = "Logic"
int S1 = 7 // size of c1
int S2 = 5
```

Q. We need to concatenate c1 and c2 into c_Arr.

NEXT PAGE

LECTURE-17 Tuesday 21/10/24

⇒ 2D - ARRAY:

no. of rows Columns
int array[3][4] = { {1,2,3,4}, {5,6,7,8}, {9,10} }
Row 1, Column 1 Row 2, Column 4
Row 1 Row 2 Row 3

* Since we haven't assigned anything in 3rd and 4th Column of Row 3; so uninitialised indexes will have '0'.

→ Printing 2D array: [Row Wise]

```
for (int i=0; i<3; i++) { → outer loop for ROWS
    for (int j=0; j<4; j++) { → inner loop for COLUMN
        cout << array[i][j];
    }
    cout << endl;
}
```

→ Printing 2D array: [Column Wise]

```
for (int i=0; i<4; i++) {
    for (int j=0; j<3; j++) {
        cout << array[j][i];
    }
    cout << endl;
}
```

* To transverse completely over 2D Array; always use 2 for loops.

* Only if we want to move on diagonals; then we can use a single for loop.

LECTURE-18 Thursday 24/10/24

⇒ Transpose of Matrix (2D array):

2	3	4
5	6	7

```
for (int i=0; i<R; i++) {
    for (int j=0; j<C; j++) {
        transpose[j][i] = Array[i][j];
    }
}
```

2	5
3	6
4	7

⇒

1	1	1	1
1	1	0	1
1	1	1	1

 Q: Search the '0' and then replace all the values in the row and column of '0' with zeros.

```
for (int i=0; i<R; i++) {
    for (int j=0; j<C; j++) {
        if (Array[i][j] == 0) {
            //replacing row with zeros
            for (int k=0; k<C; k++) {
                Array[i][k] = 0;
            }
            //replacing column with zeros
            for (int k=0; k<R; k++) {
                Array[k][j] = 0;
            }
        }
    }
}
```

i=R; j=C; } Necessary to break down again
wallu row & column bhi zero hogarhe
gi

LECTURE-19 Tuesday 29/10/24

Static

⇒ 2D Character Array: Here there shouldn't be spaces while entering characters or program won't work expectedly.

```
int main() {
    char info[5][25];
    for (int i = 0; i < 5; i++) {
        cin.getline(info[i], 25);
    }
    for (int i = 0; i < 5; i++) {
        cout << info[i] << endl;
    }
}
```

→ will print first row first, then second...

Q: char data[100] = "PF, Calculus, Phy, ICT";

Copy the data from 1D array "data" into 2D array "info" where ',' (comma) is terminating row.

```
int i = 0, r = 0, c = 0;
while (data[i] != '\0') {
    if (data[i] == ',') {
        info[r][c] = '\0';
        r++;
        c = 0;
    } else {
        info[r][c++] = data[i];
    }
    i++;
}
```

★ Always use while loop with variables like 'w' and 'c' in 2D char array since we need to manually put '\0' so it helps in that.

info[r][c] = '\0'; → This won't print garbage values. for last element jis kay baad ',' nahi hai.

LECTURE-20 Thursday 7/11/24

⇒ Vectors: Growable array

```
#include <vector>
int main() {
    vector<int> v1; // name of vector.
    cout << v1.size() << endl; // 0
    v1.push_back(23); // OR v1[0] = 23
    cout << v1.size() << endl; // 1
}
```

→ Outputting Vectors: Built-in func. that tells the length of vector.

```
for (int i = 0; i < v1.size(); i++) {
    cout << v1[i];
}
```

→ Input from User in Vector:

```
cout << "Enter the number of values you want to enter";
cin >> n;
for (int i = 0; i < n; i++) {
    cin >> v1[i];
}
```

→ can also use these two lines for input instead of above one.

```
cin >> value;
v1.push_back(value);
```

* To connect any variable from main func. to any other func., we need to pass it in parameter. (e.g: cout for outputting a variable).

M T W T F S S
Day: ☆☆☆☆☆☆

void sqrt(int a) { 'a' is parameter
int main() {
int b;
sqrt(b); 'b' is argument. }

Date □ □ □

M T W T F S S
Day: ☆☆☆☆☆☆

Date □ □ □

FUNCTIONS :

Return type name parameters
int pow (int base, int e) {
return base * e;
}

Just for concept how functions are made.

* 'int main()' does not requires 'return 0' since there is exception for 'main' functions.

* void function name () is used if the func. does not needs to return smth.

Code:

```
void PrintData() {
    cout << "Programming";
}
```

```
int main() {
    PrintData();
    return 0;
}
```

OR

Generic: int Parameter
void PrintData(a) {
 cout << a << endl;
}

```
int main() {
    int num = 78;
    PrintData(num);
}
```

// this will print 78 as 'num' is declared inside main.

* If a variable is declared outside any function/block, that is Global variable and it could be accessed from anywhere in the code after that variable is declared.

* if a variable is declared inside any function/block, that is a local variable and can only be accessed in that block.

```
void Print() { cout << data; }
int data = 3;
```

The variable 'data' is global but since it was declared after Print() func. so it would be

accessed by that func. Hence ERROR.

Imperial Notes

① Parameter passed by value:

Here the variable in PrintData() func. i.e: 'a' has to be declared as well.

② Parameter passed by Reference:

```
void Print (int &a) {
    cout << a << endl;
    a = 80;
}
```

```
int main () {
    int num = 30;
    Print (num);
}
```

this will make 'a' an alias of 'num' variable which is inside main func. But when we'll change the value of 'a' here; the value of 'num' variable in main func. will also be changed.

List of By value/reference:

"By value" main new variables are formed.

"By reference" main same variable in main func. is used as alias.

⇒ Passing 1D Array (as Parameter) in a function:

```
void Print (int a[], int size) {
    for (int i = 0; i < size; i++) {
        a[i] = i + 10;
    }
}
```

```
int main () {
    int arr [5];
    Print (arr, 5);
    for (int i = 0; i < 5; i++) {
        cout << arr[i];
    }
}
```

* imp as we can't use hard coded value even in static array cause function has to take in different arrays of diff. sizes.

Arrays are always passed "by reference" which means the changes that will be made in Print() func. will occur in the "arr" array in main.

Imperial Notes

Tuesday
12/11/24.

LECTURE-21

* To get a returned results from a void function, we can still get the result of processing inside the function by passing any variable "BY REFERENCE".

* We can only return one thing from a function.

→ To get multiple results from a function (use multiple "byREF" parameters)

```
void getSum_prod(int a, int b, int &sum, int &product) {
    sum = a + b;
    product = a * b;
}

int main() {
    int x = 5, y = 7, sum, product;
    getSum_prod(x, y, sum, product);
    cout << sum << " " << product;
}
```

The name of these two could be different.

```
void getSum_Prod(const int&a, const int&b) {
    a = 13;
    b = 12;
}
```

This will give error as using "const" in parameter only gives the variable 'a' and 'b' rights to read the value and does not allow to change it even though it is passed by reference.

* In output function of an arrays use "const" with parameter of func. as the task of func. is just to display elements and does not change it.

* Therefore the best approach is to always use parameter passed "BY REFERENCE" and use "const" if you don't want to change the value of assigned variables (and is read values).

* Also; using void func. with BYREF variables for output is more easier & simple.

```
int sum(int&a, int&b);
int main() {
    cout << sum(3, 4) << endl;
}

int sum(int&a, int&b) {
    return a + b;
}
```

Function signatures (variables optional) (Byref way variables shldve 's' in signatures).

* This allows the func. declared after "int main()" to be used in main func.

⇒ Custom Libraries:

We still can't write all the signatures before 'int main()' so we use Custom Libraries.

* To include custom library in program, we need to put inverted comma instead of angle brackets (< >).

```
#include <iostream>
```

```
#include "mylibrary.h"
```

GIST:

* Best approach is to add all your functions in a custom library (source files → new folder → header file) and then you can call those functions in main program after including that library with inverted commas ("").
(#include "mylib.h"). THE IMPORTANT thing is to add "#include <iostream>" and "using namespace std;" in custom library header file as we use "cin" & "cout" in that file.

multiple with different parameters
★ If ↑ functions have same 'name'; that is "function overloading". Now the calling of those functions will depend upon the number(s) and types of data the parameters passed.

Thursday
14/11/24

LECTURE-22

⇒ Pre-Assigned Parameters:

int getData(int a = 4, int b = 6) // Here parameters are pre-assigned.

int main() {

get Data(2, 9);
} → Even if we won't pass 2 arguments here in this function when being called; it won't cause any error and will just take a=4, b=6.

When we'll assign parameters while calling func., those arguments have higher priority and now a=2, b=9.

int getData(int x, int a = 4)

get Data(3); // "get Data()" will give error since minimum 1 argument list needs to be 1

int get Data (int x, int a = 3, int b)

static int k = 5;

This is like global variable which won't get wiped out when the handle will leave any custom function.

The Variable 'k' will survive until the program is shutdown.

Example:

```
int joinGame() {
    static int playercount = 0;
    playercount++;
}
```

```
int main() {
    for (int i = 0; i < 5; i++)
        joinGame();
}
```

→ Passing 2D array in a function as Parameter:

int Print(int arr[][No. of Columns])

int main() {

Print(array, 3);

}

No. of rows * Compiler will get info. abt. 'no. of rows' from argument and abt. 'no. of columns' from parameter.

Imp:

• Now if we'll have different 2D arrays of different column-sizes, then we can't use same custom function to print all these arrays by just passing them as argument of "Print 2D Array" function. Therefore we can use "Template":

template <size_t cols> → makes a variable named 'cols' to put in void Print2DArray(int arr[][cols], int rows) { ... } the parametered array

int main() {

Print2DArray(Arr, 5); // no. of rows

}

MTWTFSS
Day:☆☆☆☆☆☆

Date □□□

→ Template:

A function template allows you to define generic func. that can operate on different types of data (e.g.: diff. sizes or datatypes).

Another example:

```
template <typename T>
T function_name(T num, T a) {
    return num + a;
}
```

* Data 'read' & 'write' is w.r.t text file.

LECTURE-23: Tuesday 19/11/24.

⇒ File Handling:

- * Just like a variable "num" is of datatype "int"; "cout" is of type "ostream". "cin" is of type "istream".
- * cout is used for writing data on console (output window). Now to write data in memory (HDD/SSD) or into a file; we need File Handling.
- * The alternate used instead of cout is a variable (we can name it anything e.g.: 'fw') which we need to declare with type 'ofstream'.
- 'fw' and cout will do same thing but since we can use any variable name other than 'fw' to write data into file; so an extra step is just to declare that variable: ofstream fw;

* No matter how many times we use "fw" to input data into any file, it won't be printed on next line until we put "endl" or "\n". Same is the case with spaces (we manually need to give it).

MTWTFSS

Day:☆☆☆☆☆☆

⇒ Syntax: (variable → File)

Date □□□

```
## include <fstream>
```

```
# include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int data = 56;
```

```
    ofstream fw;
```

```
    fw.open("lecture23.txt");
```

```
    fw << data << endl << 32;
```

```
    fw.close();
```

```
}
```

could be of any name

OR ofstream fw("lecture23.txt");

A text file will be automatically created

* Data first needs to be stored in RAM whether we are reading from file & writing into variable/array/vector or its the other way round. Once data is in any sort of variable; then we can further use that variable to read or write data anywhere.

⇒ Data from File → Array (Read)

```
ifstream fr("res.txt");
for (int i=0; i<5; i++) {
    fr >> arr[i];
}
fr.close();
```

Array → File (Write)

```
ofstream fw("res.txt");
for (int i=0; i<5; i++) {
    fw << arr[i];
}
fw.close();
```

* Direction of double arrow shows movement of data (<< or >>).

⇒ User input into file: (write)

```
ofstream fw("File.txt");
for (int i=0; i<5; i++) {
    cin >> arr[i];
    fw << arr[i];
}
```

OR we can also use any variable instead of array.

* If we have made a folder named 'Q1' that contains .cpp file and text file; then to write the location of file in brackets; we'll write:

ifstream fr("Q1/Numbers.txt");
M T W T F S S
Day: ☆☆☆☆☆☆

Date □ □ □

M T W T F S S
Day: ☆☆☆☆☆☆

* For character arrays; fr.getline(ar, 100);
fw << ar;

→ Just use for
1D char arrays
without any loops.
Date □ □ □

⇒ Since we don't know the size of text file; so we'll use vectors with while loop and EOF.

#include <vector>

int main() {

vector<int> v1; int data;

ifstream fr("Text.txt");

while (fr >> data) { OR → while(fr.eof() == 0)

v1.pushback(data); → while(!fr.eof())

}

fr.close();

For these two, write this in the while block:
"fr >> data;"

* The above statements will move the cursor (fr) from word to word until a terminator called "eof" (end of file) is reached which is present at the end of all text files.

⇒ Appending data into a file: This will add more data at the end of a file instead of overwriting previous one.

ofstream fw("File.txt", ios::app);

* For appending, a file already needs to be existing and it won't be created automatically with the above syntax.

⇒ Check if text file is opened:

ifstream fr("File.txt");
if (!fr) {

cout << "Error opening File.txt";
return 1;
}

* For character arrays;

fr.getline(ar, 100);
fw << ar;

→ Just use for
1D char arrays
without any loops.
Date □ □ □

LECTURE-24: Thursday 21/11/24

→ Reference variable in main program:

int a = 50;

int &b = a; // This will make 'b' an alias of 'a'.

* We can make any variable of type "ostream" so we don't need to use "cout" word again and again:

Now we can use: print << "Hello";

ostream & print = cout; always

① ostream ka har variable hoga by-ref (&)

② Jab bhi by-ref variable (alias) banaiye (i.e.: print in this case), we need an existing variable jiska alias ham 'print' ko bana rahe. Therefore we will use 'cout' as the parent variable ^{so} iska alias (i.e.: print) cheezain output kar sakenge.

* 'fr' just reads that from text file until space (\n) is found.

File: 32 Alyan 4.0
Arcus: No. Name GPA

fr >> No[i] >> Name[i] >> GPA[i];

* Space say phela jitha data hoga; that will go in specific array.

File: Alyan Asif
Ali Ahmed
Taha Javed.

while (getline(fr, data)) {
cout << data << endl;
}

* So the above code will read the entire line from file at once instead of only data before spaces as we are using "getline".

And 'fr' is used instead of 'cin' since we are reading data from file instead of user input.

NOT IN SYLLABUS.

⇒ Fetch data from string Array and store separately in different types of array;

```
#include <stringstream>
while (getline(fr, data)) {
    stringstream sr(data); // making variable 'sr' of type 'stringstream'
    sr >> roll[i] >> name[i] >> GPA[i]; // that reads from 'data'
    i++;
}
```

→ This will stop putting values in arrays as soon as it catches a space or newline.

NOT IN SYLLABUS

DYNAMIC ARRAY:

```
int size;
cin >> size;
int *Arr = new int[size];
```

Arr is pointer to integer

Stores in Heap (not stack) part of memory. Tells the 'heap' in memory to make the memory blocks of 4 bytes (as 'int')

* Vector khud grow krta hai and dynamic array apne size main variable le skta hai instead of constant

LECTURE-25

Tuesday
26/11/24.

⇒ Delimeter: The separator jahan tak hamain data read getline (fr, s, ',');

The data from a text file will be read until a comma (,) appears. Then it will store it in string variable 's'.

Cin.getline(arr, 10)

- This array is character array. (only this could be used for char array input)

- Needs a length

- Does not need #include <string>

- Stops taking input as 'Enter' is pressed (when it finds '\n')

Getline(cin, arr)

- This is either string array or string variable

- Does not need a length

- Needs #include <string>

- Stops taking input as 'Enter' is pressed. (when it finds '\n')

⇒ POINTERS: Stores Hexadecimal. The address of any memory block.

```
int x;
int *ptr; ptr is a pointer to integer.
```

* Pointers does not have any datatype since it store Hexadecimal values.

ptr = &x; takes the memory location of 'x' and store in 'ptr'

* 'Ptr' will only store the address of an 'Integer' variable in this case.

cout << *ptr

Dereferencing: Targets the data of the address stored in ptr for cout or cin.

cout << ptr

→ This will just print Hexadecimal address.

Data	0	1	2	3
	20	30	40	50
	C0	C4	C8	DC

`ptr = &data[2];` // store C8
`ptr++;` This increment '4' since the type of `ptr` is `int`.
~~and size of `int` is 4 bytes.~~
`cout << *ptr;` // Now it prints data in 'DC' (i.e. 50)

`ptr = 70; X`
`*ptr = 70; ✓` // stores '70' on the address stored in `ptr`.

~~int *p1; int u;
 p1 = &u;
 *p1 = 10; *p1~~
~~int *ptr; // declares pointer
 int u;
 ptr = &u; // stores memory in pointer
 *ptr = *ptr + 10; // updates the value
 ptr++;~~

GIST:

`int *ptr;` // declares pointer
`int u;`
`ptr = &u;` // stores memory address of a variable in pointer
`*ptr = *ptr + 10;` // updates the value of variable 'u' (which pointer is pointing)
`ptr++;` // increment the value of pointer (by +1 in this case of 'integer')

⇒ Char Array and file handling

`char arr[6];`
`ifstream read("test.txt");`
`for (int i=0; i<6; i++) {`
`read >> arr[i];`
`}`

test.txt:
 abcde

char arr.

0	1	2	3	4	5
a	b	c	d	e	\0

eventhough ">>" reads the data until it caters space or next line and then stores it on a single index of an array, the data "abcde" won't be stored on `arr[0]`. Since this is a character array, so it will automatically break "abcde" into ^{single} characters & store in single index.

`string array[6];`
`ifstream read("test.txt");`
`for (int i=0; i<6; i++) {`
`read >> arr[i];`
`}`

String array

0	1	2	3	4	5
abcde					